Routes to Safety

An evacuation-services map from open geospatial data — datasets, tiles, routing, rendering

DuckDB · Overture Maps · Planetiler · PMTiles · MapLibre GL JS · OSRM

Vishal Kumar · Shaishav Maisuria

FOSS4G NA 2026, Sacramento — Monday 2 November, 09:00–12:00, Room Beavis

github.com/vishal4c/routes-to-safety-workshop

What we work with — open datasets and open tools

OPEN DATASETS

OpenStreetMap
ODbL

the road network we route on — one Geofabrik .osm.pbf extract

Overture Maps
ODbL / CDLA-P 2.0

roads, places, towns, water, land cover, buildings — five themes, staged as GeoJSON

CAL FIRE FRAP
CC-BY

Fire Hazard Severity Zones and fire perimeters (ArcGIS feature services)

ESA WorldCover
CC-BY 4.0

land cover at 10 m, via the Overture base theme

OPEN TOOLS — one per phase

DuckDB ✦ spatial
Phase 1

SQL over GeoJSON, and over Overture Parquet on S3 — a county out of the planet in seconds

Planetiler → PMTiles
Phase 2a

GeoJSON → vector tiles → one static file served by HTTP Range requests

OSRM
Phase 2b, 2c, 3c

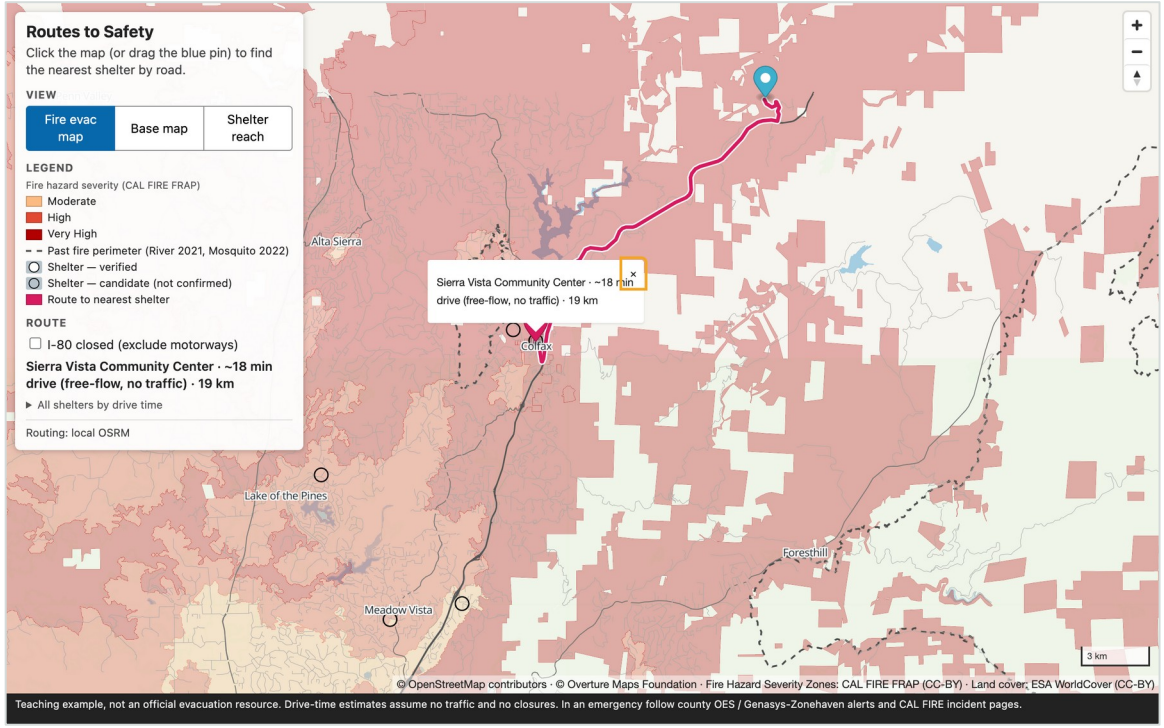
routing graph from the OSM extract; /nearest, /table, /route → nearest by road, reach bands, closures

MapLibre GL JS
Phase 3

draws the tiles from a JSON style — three views, routing wired into the page

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

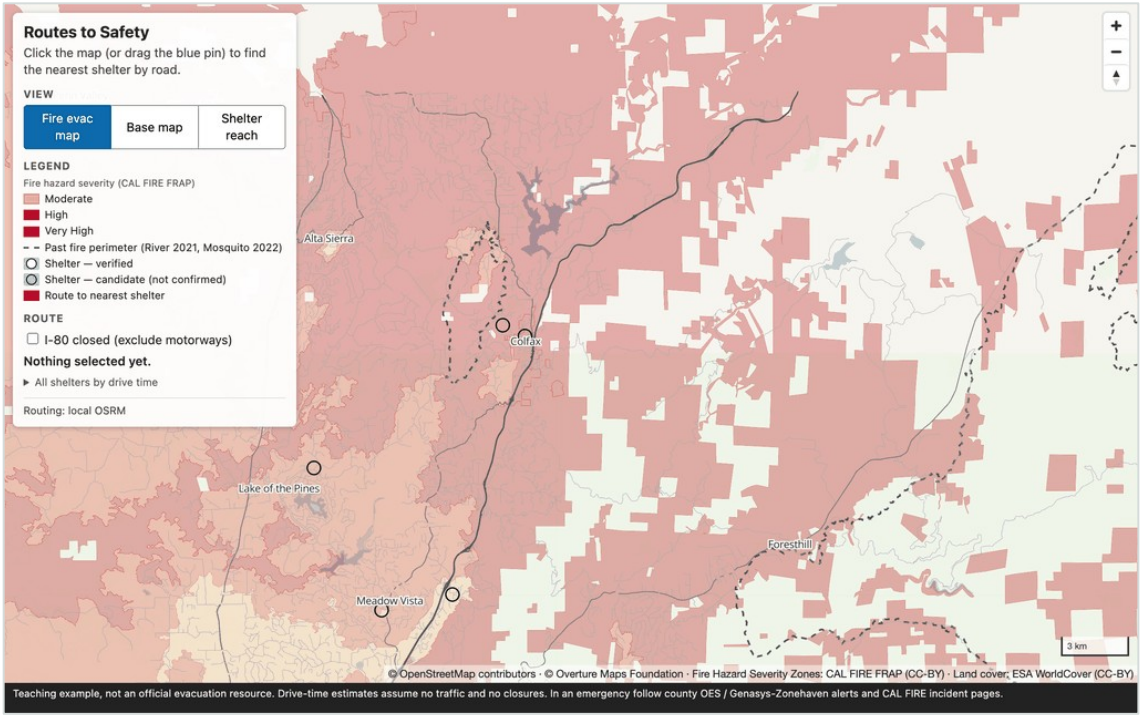
What we will have built



Public demo: vishal4c.github.io/routes-to-safety-workshop/demo/ — Placer County, I-80 corridor

- **Our own vector tiles** — roads, water, hazard, perimeters, labels, shelters: one 6.5 MB file
- **Three views, one at a time** — Fire evac map, Base map, Shelter reach
- **Live routing in every view** — click anywhere → nearest shelter by road
- **'I-80 is closed'** — one parameter, a new answer (calculation, not detection)
- **Shelter reach** — 10 / 20 / 30-minute bands from one /table call
- **Static files only** — no database, no tile server; works offline

The app in motion



Animates in slideshow mode. Recorded against the local OSRM container; the public demo routes via router.project-osrm.org, without the closure toggle.

1 Drop a pin at Dutch Flat → Sierra Vista CC, 18 min

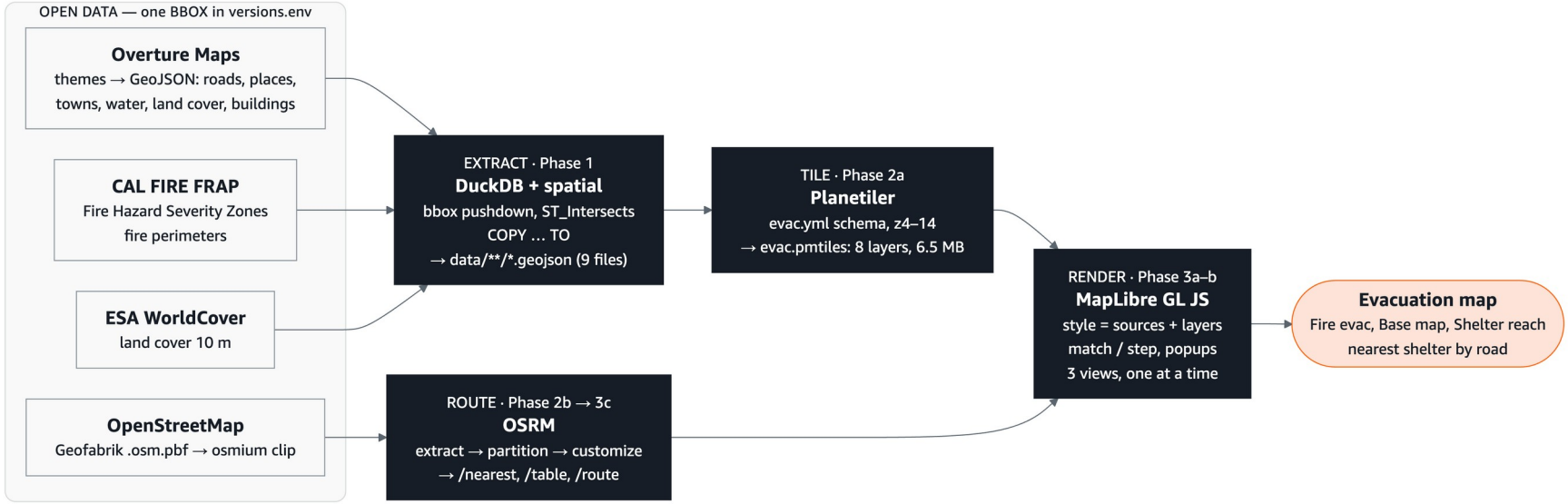
2 Tick "I-80 closed" → same shelter, 31 min via Dutch Flat Rd

3 Foresthill: nearest on the map (Georgetown, 12 km) ≠ nearest by road (Placer High, 36 min)

4 Shelter reach view → 10 / 20 / 30-min bands; click Placer High → only its reach

5 Base map view → a plain street map; the pin and the route stay — routing works in every view

The pipeline: one extract → a basemap and live routes



Phase 4 change one bbox line in versions.env → run the nine scripted steps → the same map for a town of our choice

Pinned	DuckDB 2.0.0	Planetiler 0.10.2	pmtiles 1.31.2	MapLibre GL JS 6.14	OSRM v26.10.0	verified by CHECKSUMS.sha256

Agenda — five phases, one map

0 **Setup**
Pick a way to run, then one checkpoint line — the same green line on every screen
DuckDB

1 **Data prep**
Explore the staged data, pick candidate shelters against the hazard, export GeoJSON
DuckDB · Overture Maps

Break

2 **Map artifacts**
Build the tiles; the routing graph and the reach bands are prebuilt
Planetiler · PMTiles · OSRM

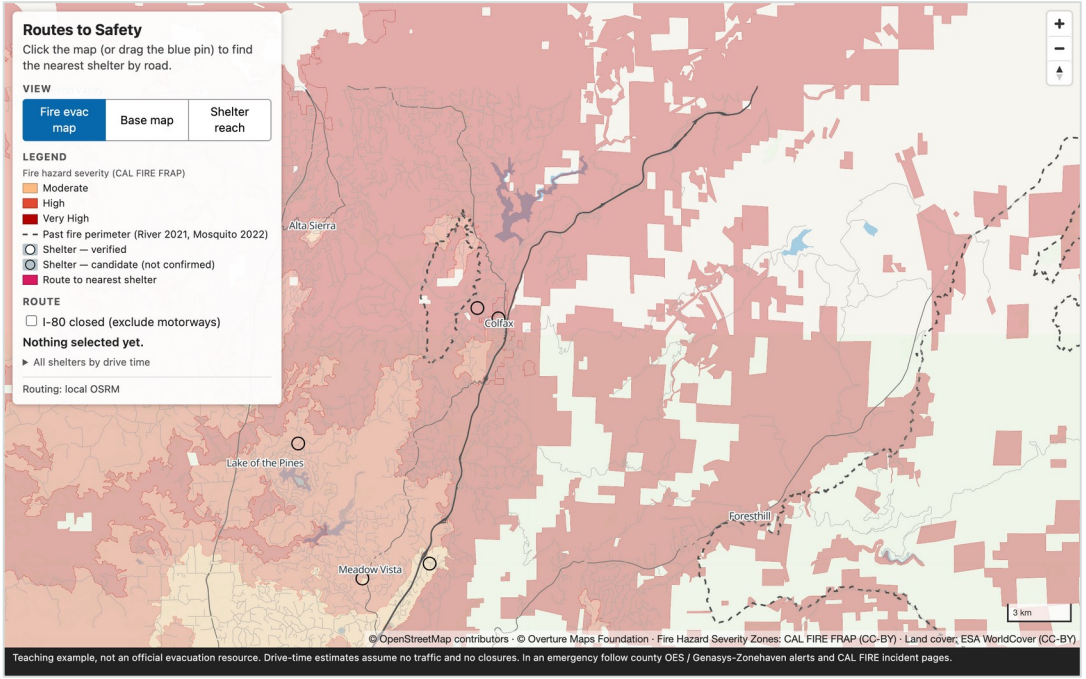
3 **Application**
Style the map, switch between three views, wire in routing, close I-80
MapLibre GL JS · OSRM

4 **Adapt**
One bbox in versions.env, the same nine steps for a town of our choice
versions.env · scripts/

Sticky note on the lid: **green** = fine, **red** = stuck, come find us. Finished early? Every phase card has an extra exercise.

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

Study area: the Placer County I-80 corridor



Shelters: 1 verified (Sierra College) + 9 candidates from Overture places. Hazard: CAL FIRE FHSZ 2024 (SRA + LRA). Teaching example, not an official evacuation resource. Drive-time estimates assume no traffic and no closures.

Rocklin → Auburn → Colfax → Dutch Flat / Foresthill

- bbox `−121.40, 38.72, −120.52, 39.20` (W, S, E, N) — about 0.4 square degrees
- 36,294 drivable road segments, 10 shelters

Why here

- River Fire, Aug 2021 — Colfax evacuated for ~2 days
- Mosquito Fire, Sep 2022 — the shelter moved mid-incident to Sierra College: the safe place is not a fixed point
- One interstate, steep canyons — nearest on the map is often not nearest by road

What we leave out

- Shelter status — open, closed or full
- Drive time ≠ evacuation time
- Live road closures
- Shelter capacity

PHASE 0

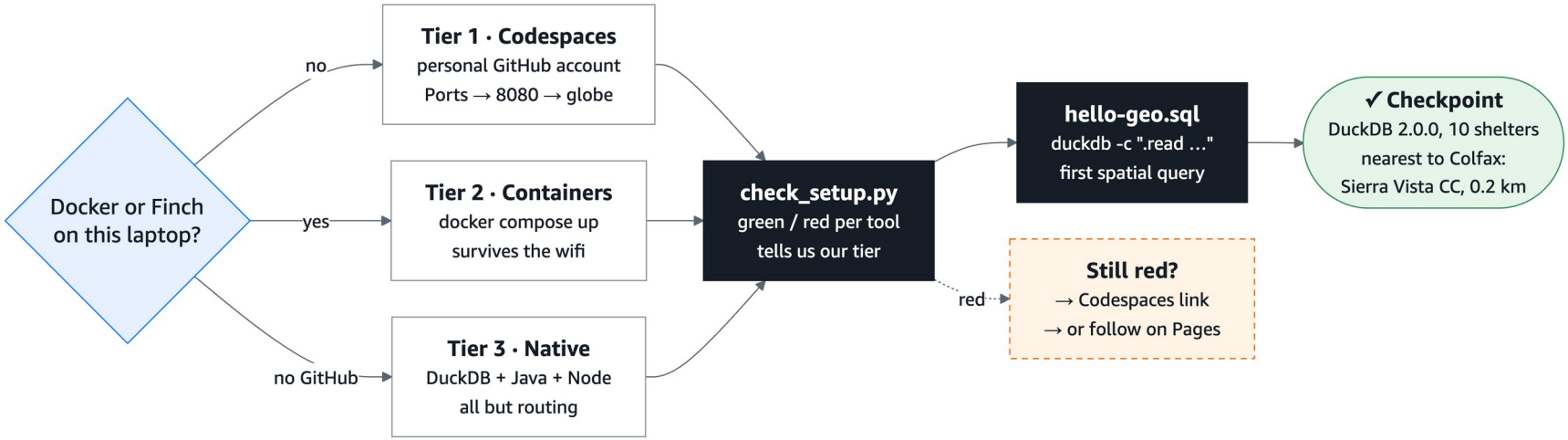
Setup

Welcome → **pick a way to run** → **setup checkpoint**

DuckDB 2.0.0 | Codespaces | Docker / Finch | native

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

Phase 0 in one picture — pick a way to run, run two lines, hands up



Phase 0 Docker or Finch on the laptop? → Codespaces, containers or native → two commands → hands up

✓ **Checkpoint** OK — DuckDB 2.0.0 · 10 shelters (1 verified, 9 candidate) · nearest to Colfax (crow-flies): Sierra Vista Community Center, 0.2 km

Three ways to run — pick one, keep a neighbour

Tier	Who	Routing	Notes
1 Codespaces	no Docker on the laptop	yes	**personal** GitHub account; sit next to a Docker user port 8080 via the Ports tab ⟶ globe icon
2 Local containers	**recommended** where Docker / Finch is available	yes	survives wifi loss corporate laptops: `finch compose up`
3 Native	DuckDB + Java + Node, no containers	no	Windows: Git Bash map + shelters only — OSRM ships no native Windows / macOS binary
4 Browser only	laptop failed	no	Pages site: follow along; the demo routes via the public OSRM server

PHASE 0 · setup checkpoint

Run both lines from the repo root; hands up when the output matches the line on the right.

Still red? Codespaces link. Blocked? Pages link, follow along.

```
python3 setup/check_setup.py
duckdb -c ".read setup/hello-geo.sql"
```

```
OK — DuckDB 2.0.0 · 10 shelters (1 verified, 9 candidate)
  · nearest to Colfax (crow-flies): Sierra Vista Community Center, 0.2 km
```

PHASE 1

Data prep with DuckDB

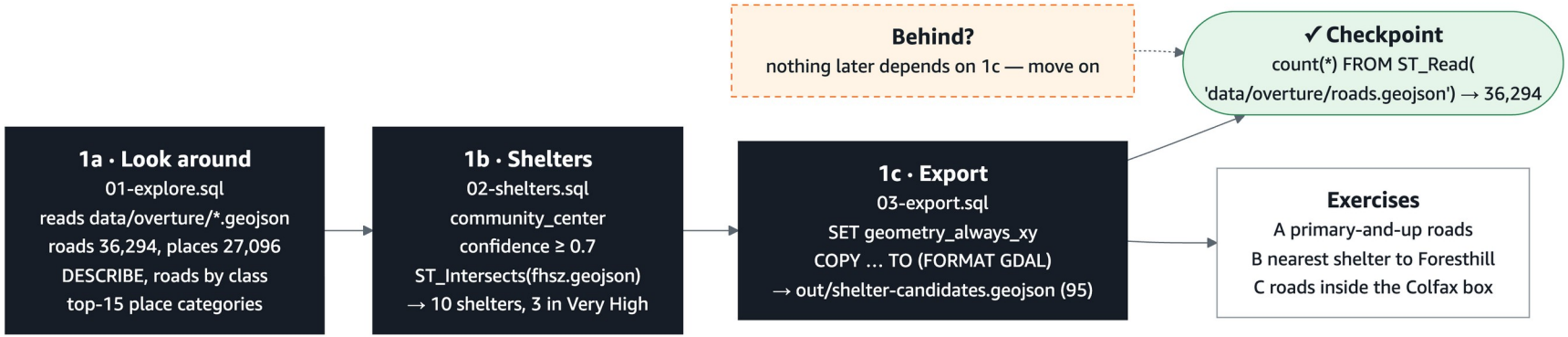
1a look around → **1b candidate shelters** → **1c export** → **exercises**

DuckDB 2.0.0 + spatial

Overture Maps 2026-09-23.1

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

Phase 1 in one picture — three SQL files, one folder of GeoJSON



Phase 1 1a explore → 1b candidate shelters → 1c export

✓ **Checkpoint** SELECT count(*) FROM ST_Read('data/overture/roads.geojson') → **36,294**

Overture Maps in one slide

Open map data from the Linux Foundation, released monthly as **GeoParquet** in five themes: `transportation`, `places`, `base`, `buildings`, `divisions`

Built from OpenStreetMap plus other sources; licence differs by theme (ODbL / CDLA-Permissive 2.0)

Parquet stores a `bbox` struct per row → DuckDB skips every file and row group outside the rectangle. Planet → Placer corridor without downloading the planet; we stage the result as GeoJSON in data/overture/.

Attribution on every published map: © OpenStreetMap contributors, © Overture Maps Foundation

36,294 drivable roads	**27,096** places
121 towns (divisions)	**1,920** water polygons
2,564 land-cover polygons	**12,606** buildings (Auburn)

Our study area, release 2026-09-23.1 — fetched to GeoJSON by scripts/fetch-overture.sh in 24 s

SQL in three lines, then one spatial predicate

```
LOAD spatial;
SELECT class, count(*) AS n      -- what do I want back?
FROM ST_Read('data/overture/roads.geojson')
GROUP BY class ORDER BY n DESC;  -- one row per group
```

No GIS yet — this is just a table.

- `SELECT` what do we want back
- `WHERE` which rows
- `GROUP BY` one count per group

Results to expect

class	n	class	n
residential	25,995	primary	1,305
tertiary	4,333	motorway	541
secondary	2,792	trunk	63

```
LOAD spatial;
SELECT s.name FROM shelters s, fhsz h
WHERE ST_Intersects(s.geom, h.geom)   -- yes / no
  AND h.FHSZ_Description = 'Very High';
```

A predicate is a yes/no question about two shapes.

It goes in WHERE like any other condition.

Phase 1b answer: three shelters sit in a Very High zone — Colfax High School, Sierra Vista Community Center, Georgetown School.

DuckDB 2.0 gotcha: `SET geometry_always_xy = true;` right after `LOAD spatial;` or the GeoJSON writer swaps lat/lon.

Columns cheat sheet — raw Overture vs. our GeoJSON

We want	Raw Overture (S3)	Our data/overture/*.geojson
a place's name	`names.primary`	`name`
a place's category	`taxonomy.primary`	`category`
how sure Overture is	`confidence` (use >= 0.7)	same
road type	`class`, `subclass`	same
the shape	`geometry`	same

`scripts/fetch-overture.sh` already flattened these — today we type the right-hand column.

`categories` was removed in Sep 2026 → `taxonomy.primary`

There is no "fairground" category — Gold Country Fairgrounds hides under another one.

Writing GeoJSON: `SET geometry_always_xy = true` first, or lat/lon come out swapped.

The noise is the lesson: John L Sullivan Chevrolet is a community_center at confidence 0.92.

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

PHASE 1 · hands-on — data prep with DuckDB

1a · Look around

`duckdb -c ".read 00-data/01-explore.sql"`

three queries — DESCRIBE roads, roads by class, top-15 place categories; the expected output is printed in the file

1b · Candidate shelters

`duckdb -c ".read 00-data/02-shelters.sql"`

community_center + confidence ≥ 0.7 ⟶ spot the Chevrolet dealer ⟶ open data/shelters.geojson ⟶ ST_Intersects with FHSZ

1c · Export

write GeoJSON with SQL (right):

`duckdb -c ".read 00-data/03-export.sql"` ⟶ out/shelter-candidates.geojson, 95 places

Nothing later depends on 1c — behind? move on.

```
LOAD spatial; SET geometry_always_xy = true;
COPY (SELECT id, name, category, confidence, geom
      FROM ST_Read('data/overture/places.geojson')
      WHERE category IN ('school','community_center', … ))
TO 'out/shelter-candidates.geojson' (FORMAT GDAL, DRIVER 'GeoJSON');
```

Exercises

A count motorway + trunk + primary ⟶ 1,909

B crow-flies nearest shelter to Foresthill (−120.8182, 39.0199) ⟶ Georgetown School, 12.4 km. **Remember it for Phase 3c.**

C (finish early) count the roads inside the Colfax box with ST_Intersects

✓ **Checkpoint** `duckdb -c "LOAD spatial; SELECT count(*) FROM ST_Read('data/overture/roads.geojson')"` ⟶ **36,294**

PHASE 2 → 3

Map artifacts → the application

2a build the tiles → **3a serve & style** → **3b three views**

Planetiler 0.10.2

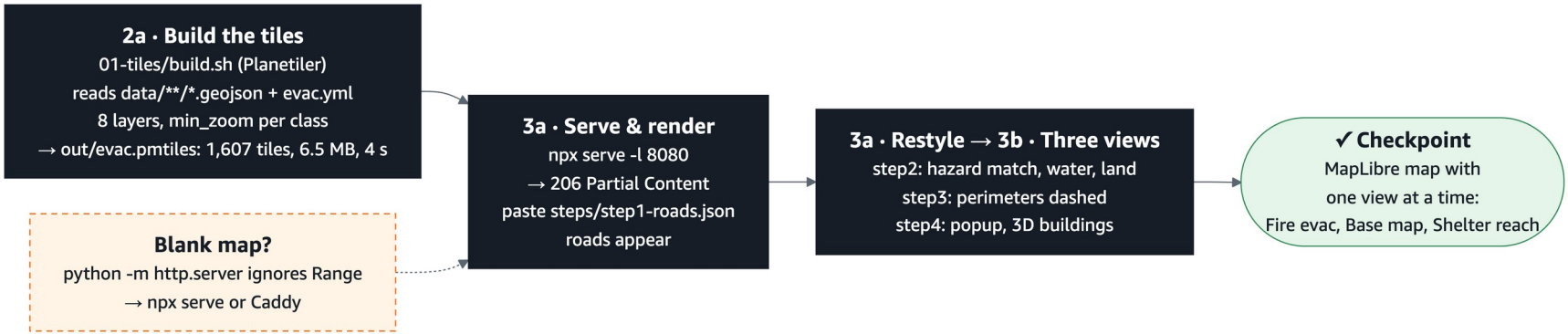
go-pmtiles 1.31.2

MapLibre GL JS 6.14

pmtiles.js 4.5

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

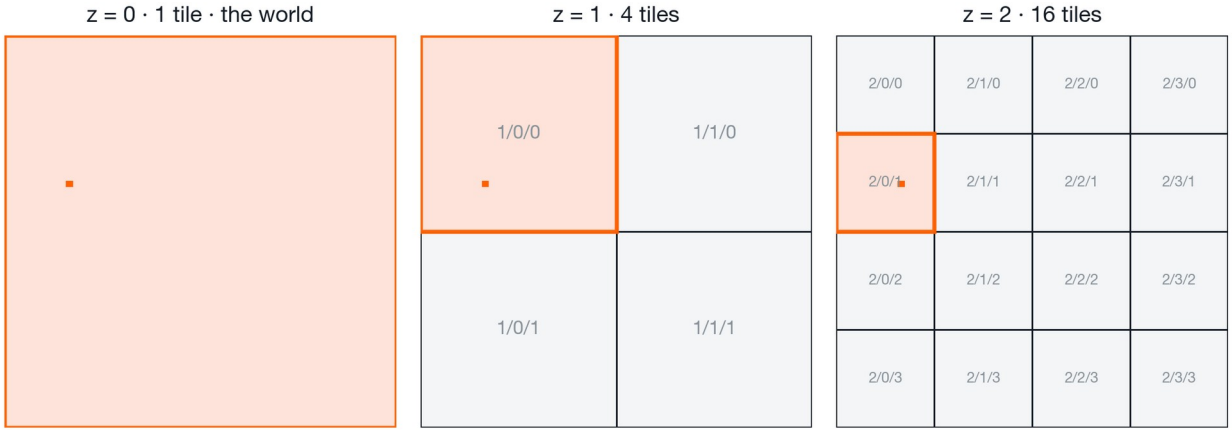
Phase 2 ⟶ 3 in one picture — GeoJSON ⟶ one tile file ⟶ a styled map



Phase 2a ⟶ 3b 2a build the tiles ⟶ 3a serve, render, restyle ⟶ 3b three views

✓ **Checkpoint** curl -sI -H "Range: bytes=0-15" http://localhost:8080/out/evac.pmtiles ⟶ **206 Partial Content**

Why tiles: the map is cut into a z/x/y quadtree



z	tiles on Earth	over our bbox
4	256	1
8	65,536	4
10	1,048,576	12
12	16.8 M	88
14	268 M	1,218

Each zoom splits every tile into four (a quadtree). A tile is addressed z/x/y and holds only what falls inside it — the browser fetches the ~12 tiles on screen, never the whole dataset. Our file: 1,607 tiles, z4–14, 6.5 MB, one file.

Each zoom level splits every tile into four. z0 is the whole world in one tile; by z14 the planet is 268 million tiles — but our rectangle touches only 1,218 of them. The browser fetches the handful under the viewport, at the zoom it is showing, and nothing else.

That is why one 6.5 MB file replaces a 17 MB GeoJSON and a database: 1,607 tiles for z4–14, each holding only the geometry that matters at that zoom (motorways from z6, residential streets from z10). Zoom in → a new, smaller square; pan → the neighbour square.

Vector tiles, PMTiles and the Range-request trap

A browser cannot draw 36,294 roads from a 17 MB GeoJSON fast.

So we cut the map into a z/x/y grid. Each tile holds geometry and attributes, not pixels — the browser draws it, so restyling needs no rebuild.

PMTiles = every tile in one file with an index. The browser asks for byte ranges: no tile server, any static host (GitHub Pages, S3, a USB stick).

The trap: `python -m http.server` ignores Range headers ⟶ the map stays blank. Use Caddy (:8080) or `npx serve -l 8080 .` and look for **206 Partial Content**.

```
# the only test needed
curl -sI -H "Range: bytes=0-15" http://localhost:8080/out/evac.pmtiles | head –1
HTTP/1.1 206 Partial Content
```

```
# 01-tiles/evac.yml  (excerpt — 8 layers, all local files, zero network)
sources:
  roads: { type: geojson, local_path: data/overture/roads.geojson }
  towns: { type: geojson, local_path: data/overture/towns.geojson }
layers:
  - id: roads
    features:
      - { source: roads, geometry: line, min_zoom: 6,
          include_when: { class: [motorway, trunk, primary, secondary] } }
      - { source: roads, geometry: line, min_zoom: 10, min_size: 1,
          exclude_when: { class: [motorway, trunk, primary, secondary] } }
```

MapLibre style: sources, layers, and two expressions

```
"fill-color": ["match", ["get", "FHSZ_Description"],
  "Moderate",  "#fdbb84",
  "High",      "#e34a33",
  "Very High", "#b30 000",
  "#cccccc"]                    // fallback
```

```
"line-width": ["step", ["zoom"], 0.6, 10, 1.2, 13, 1.6]
```

`match` — one output per category (hazard class ⟶ OrRd)

`step` — one output per range (zoom ⟶ width)

Case and spelling must match exactly: "Very High", not "very_high". FHSZ is also there as 1 | 2 | 3.



A style is JSON: `sources` (our one PMTiles file) + `layers` (one per thing drawn, bottom to top).

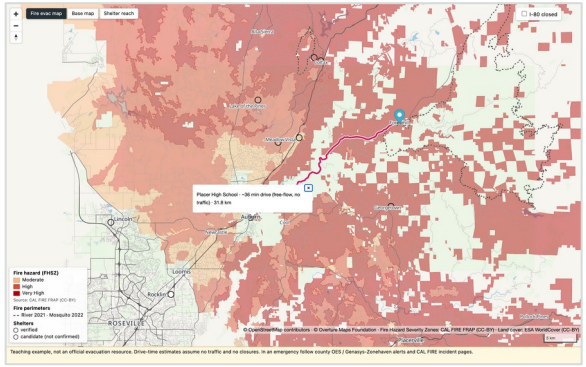
Paste-in steps: index.html sorts pasted layers into a fixed order and de-duplicates by id — paste order never matters.

Labels: a symbol layer with text-field + text-font; glyphs vendored in 02-web/fonts, so labels work offline.

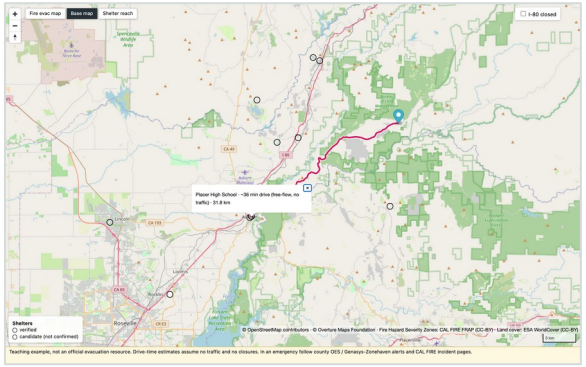
Three views, one at a time: setLayoutProperty(visibility) over a list of layer ids, legend swapped with it — no style reload, so routing keeps working.

Shelter reach: a GeoJSON source + setFilter(shelter_id); the 10 / 20 / 30-min rings never overlap, so they stay translucent over the roads. Precomputed in Phase 2c.

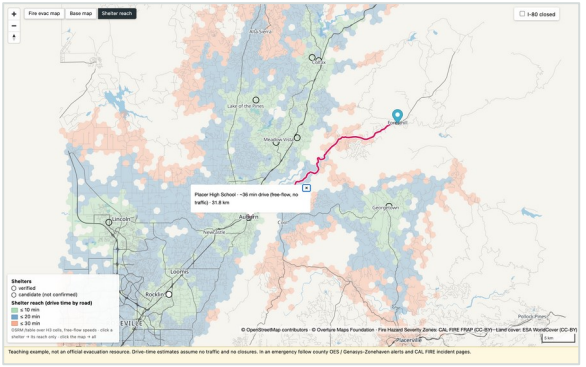
What Phase 3 produces: three views of the same tiles



Fire evac map — our tiles: FHSZ Moderate / High / Very High, dashed perimeters, city labels, shelters, route



Base map — a standard street raster (Esri, no API key); only shelters, the pin and the route on top (needs internet)



Shelter reach — 10 / 20 / 30-minute bands (green, blue, light red) from every shelter; click a shelter for its own bands

Strictly one view at a time

- the legend follows the view; reach bands are never drawn over the hazard fill
- hazard edges are zone boundaries, not a guarantee — a road just outside a polygon is not "safe"

✓ **Phase 3a checkpoint**

- Very High zones solid red, water blue
- 10 shelter circles, 1 filled
- Mosquito perimeter dashed, city labels on
- attribution visible bottom-right

PHASE 2a ⟶ 3b · hands-on — build and style the map

Open the page

Codespaces Ports tab ⟶ 8080 ⟶ globe

Docker / Finch http://localhost:8080/02-web/

Native `npx serve -l 8080 .` ⟶ same URL

2a · Build the tiles

`01-tiles/build.sh` `# = java -jar planetiler.jar generate-custom --schema=01-tiles/evac.yml --output=out/evac.pmtiles`

`pmtiles show out/evac.pmtiles`

Expected: 1,607 tiles, zoom 4–14, 8 layers, ≈ 6.5 MB, ~4 s

No PMTiles file? ⟶ `cp data/prebuilt/evac.pmtiles out/`

3a · Serve & render (step 1)

in 02-web/index.html find

`// STEP 1: paste here` ⟶ paste steps/step1-roads.json ⟶ reload; uncomment AttributionControl ⟶ grey roads + attribution bottom-right

3a · Restyle (steps 2–4)

step2: hazard match (OrRd), water fill, land cover, city labels; shelters filled = verified, hollow = candidate

step3 (if time): fire perimeters dashed grey

step4 (finish early): click popup with the name; fill-extrusion on buildings (zoom ≥ 13, tilt)

stuck? ⟶ paste steps/step3-styled.json; totally lost ⟶ open 02-web/solution.html

✓ **Checkpoint** Very High solid red, water blue, 10 shelters (1 filled), Mosquito perimeter dashed, attribution visible

3b · Three views — try the Base map and Shelter reach buttons, one view at a time: a plain street map, or each shelter's 10 / 20 / 30-min drive-time bands (click a shelter to isolate it); then come back, it is the same page.

PHASE 3c

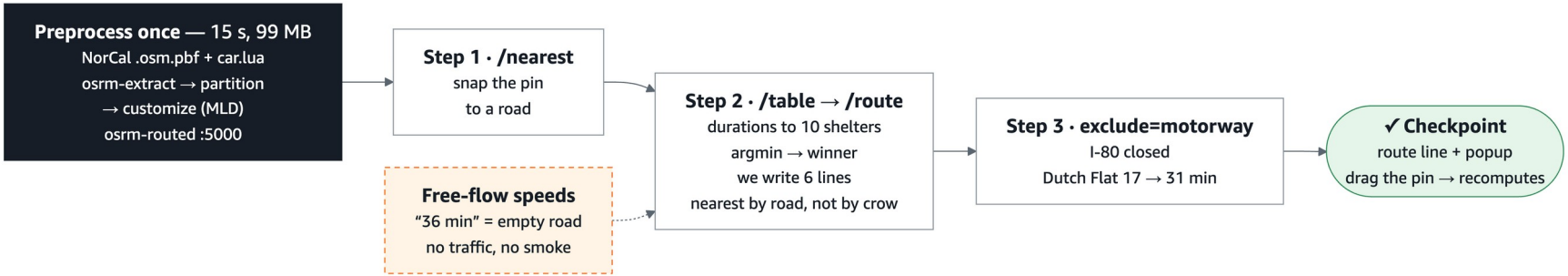
Routing in the application (OSRM)

the routing graph (2b) → **/nearest** → **/table → argmin → /route** → **I-80 closed**

OSRM v26.10.0 (MLD) | tiers 1–2 route | tiers 3–4 pair up

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

Phase 3c in one picture — preprocess once, then three HTTP calls

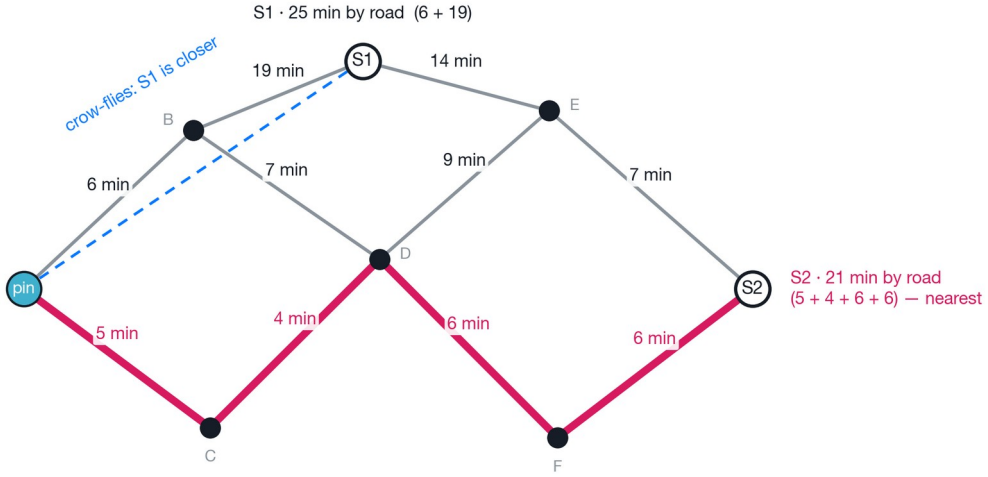


Phase 3c hello route (/nearest) → nearest by road (/table → argmin → /route) → I-80 closed (exclude=motorway)

Teaching numbers

- Dutch Flat → Sierra Vista CC: 17.1 min, 31.3 with I-80 closed
- Foresthill → Placer High: 35.8 min, 31.8 km

Why a routing graph: nearest on the map ≠ nearest by road



Nodes = intersections · edges = road segments · weight = seconds at free-flow speed. Dijkstra grows the cheapest frontier from the pin until it touches a shelter. OSRM precomputes this once (extract → partition → customize, 15 s, 99 MB) so every /route or /table answer is milliseconds.

Edge weights = length ÷ free-flow speed. Pink: the cheapest path pin → C → D → F → S2. Grey dashed: the straight line the map suggests.

The crow lies

S1 is closest on the map, 25 min by road. S2 looks farther but is 21 min: a canyon, a river or a missing bridge bends the real distance.

A road network is a graph

nodes = intersections, edges = road segments weighted in seconds. Dijkstra grows outward from the pin, cheapest node first, until it has settled every shelter.

Why preprocess

on 36,294 roads a search per click is fine; on a region it is not. OSRM builds the graph once (15 s, 99 MB), then answers in milliseconds.

Build-once artifact

like the tiles: made in Phase 2, used by the app in Phase 3c.

Routing in three minutes, and three HTTP calls

A road map is a graph: intersections are nodes, segments are edges, each edge costs seconds (length ÷ free-flow speed).

A route is the cheapest path — Dijkstra explores outward until it reaches the target.

Too slow at scale → OSRM preprocesses once: **MLD** partitions the graph into cells and precomputes costs between cell borders. Our graph: 15 s to build, 99 MB.

Weights are free-flow speeds, not traffic. "36 minutes" means 36 minutes on an empty road.

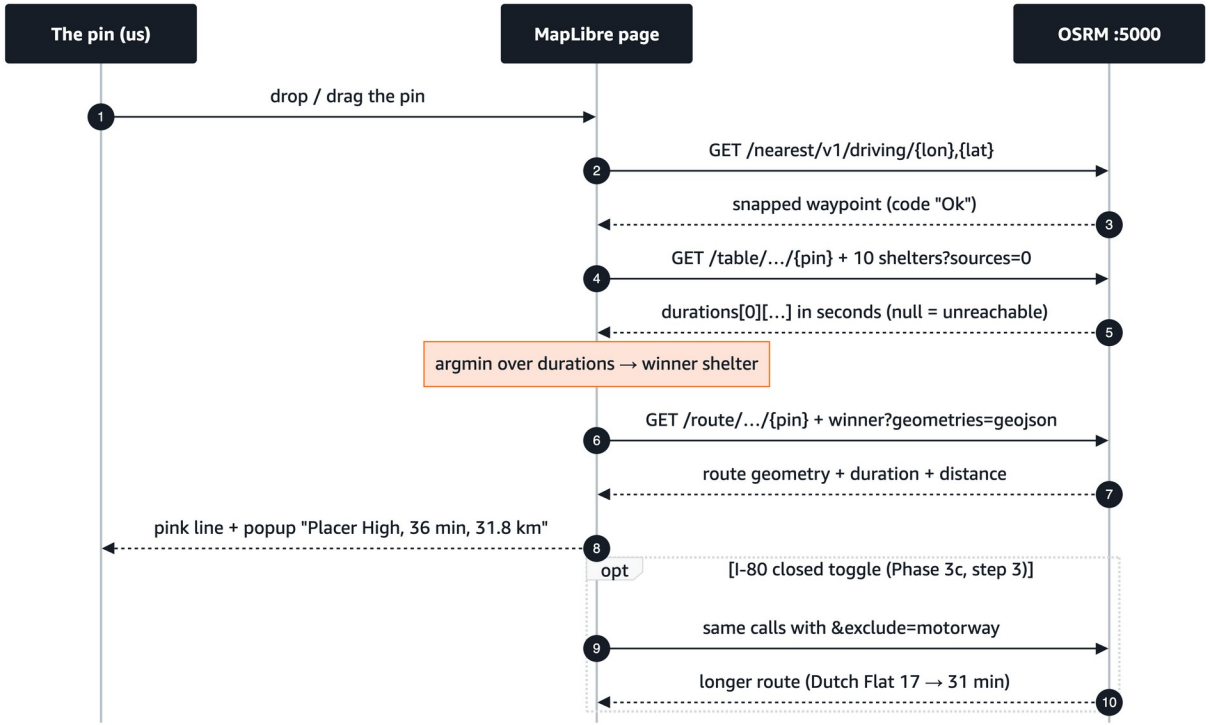
Call	Question it answers
/nearest	Which road point is closest to this coordinate? (snap)
/route	Path + duration + distance between two points
/table	Durations (and distances) from many points to many points

```
GET $OSRM_URL/{service}/v1/driving/{lon},{lat};{lon},{lat}   # longitude first!
```

Tier	$OSRM_URL
1 Codespaces	<forwarded-8080-URL>/osrm (Ports → globe; never forward 5000)
2 Local Docker / Finch	http://localhost:8080/osrm (Caddy strips /osrm → osrm:5000)
3 Native, 4 Browser	no routing — pair with a neighbour

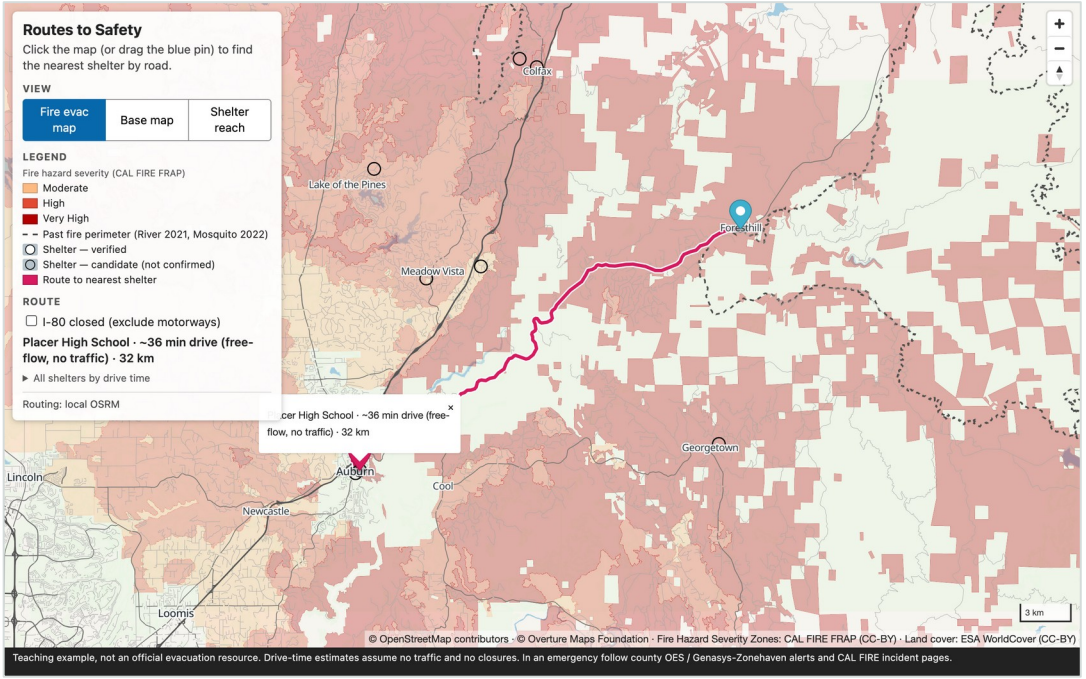
Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

What the browser and OSRM say to each other



Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

Nearest on the map ≠ nearest by road



Demo at Foresthill (–120.8182, 39.0199): ranked list from one /table call, route from /route.

Foresthill sits on a ridge above the American River canyon.

Exercise B (crow-flies): Georgetown School, 12.4 km

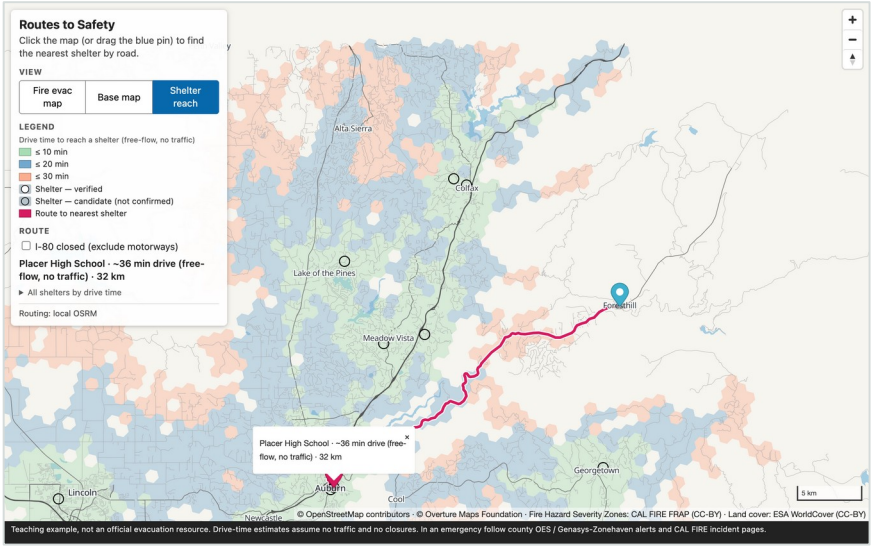
Phase 3c (/table, by road): Placer High School, 35.8 min / 31.8 km. Georgetown is 59.7 min / 54 km by road.

A ridge, a river canyon or a missing bridge makes the "close" shelter far. This is why the shelter list is ranked by /table, not by ST_Distance.

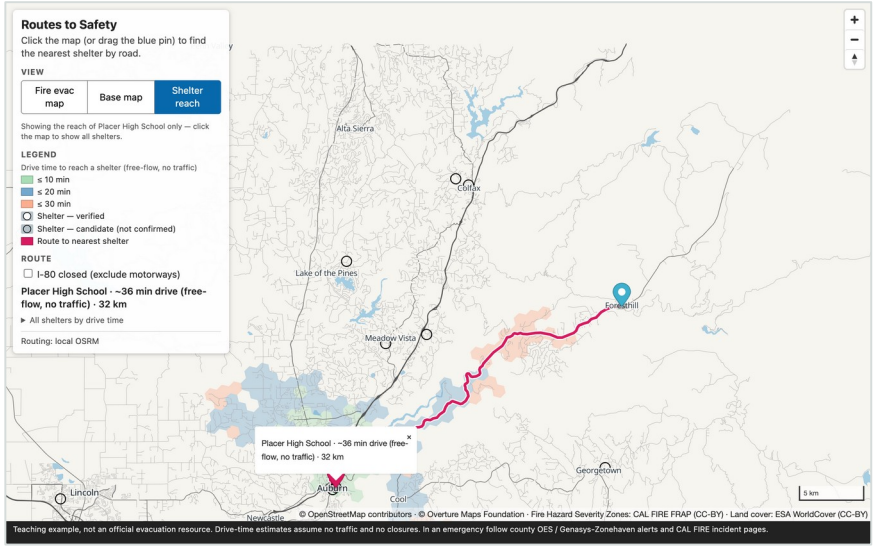
How the graph was built: `osmium extract -b <bbox> norcal-latest.osm.pbf ⟶ osrm-extract -p car.lua ⟶ osrm-partition ⟶ osrm-customize ⟶ osrm-routed --algorithm mld --max-table-size 500`

Finish early? build the graph by hand: `scripts/build-osrm.sh data/osm/colfax.osm.pbf`

From routes to coverage: /table once, draw the reach



Shelter reach view, all shelters — 10 / 20 / 30-minute bands in green, blue, light red (dissolved H3 cells)



Click a shelter ⟶ only its 10 / 20 / 30-minute reach

`scripts/make-coverage.py`: H3 res-8 grid over the bbox (3,341 cells with a road) ⟶ one batched `/table` call (cells × 10 shelters) ⟶ DuckDB dissolves the cells into one band per shelter. Stretch A in Phase 3c is the 400-point version with `grid.sql`.

PHASE 3c · hands-on — routing

Step 1 · Hello route

`curl "$OSRM_URL/nearest/v1/driving/-121.08,38.90"` (PowerShell: curl.exe)

Expected: JSON with "code":"Ok" and a waypoints array. Then /route…? geometries=geojson; paste via steps/step1-route.js → pink line Foresthill → Placer High, console "Route: 35.8 min, 31.8 km".

Step 2 · Nearest shelter by road

in steps/step2-scaffold.js

we write 6 lines: /table URL → argmin durations[0] → /route → setData → popup. Compare with Exercise B. Drag either marker.

not routing? → open steps/step3-solution.js

Step 3 · I-80 is closed

add the

`&exclude=motorway` toggle; start from **Dutch Flat (-120.838, 39.198)**: 17 → 31 min to Sierra Vista CC, the route leaves I-80 for Dutch Flat Rd / Rollins Lake Rd.

Real closures aren't a parameter — nobody flags a downed power line before someone drives into it. This shows the calculation, not detection.

Tiers 3/4: no routing by design — pair with a Docker/Codespaces neighbour or follow the screen.

Finish early? Stretch A — 15-min reach: grid.sql → one /table POST (400 points)

Stretch B — build the graph for Colfax; finish at home, the repo has the solution

✓ **Checkpoint** drag anywhere → line + popup update; the toggle changes the route; footer disclaimer visible

PHASE 4

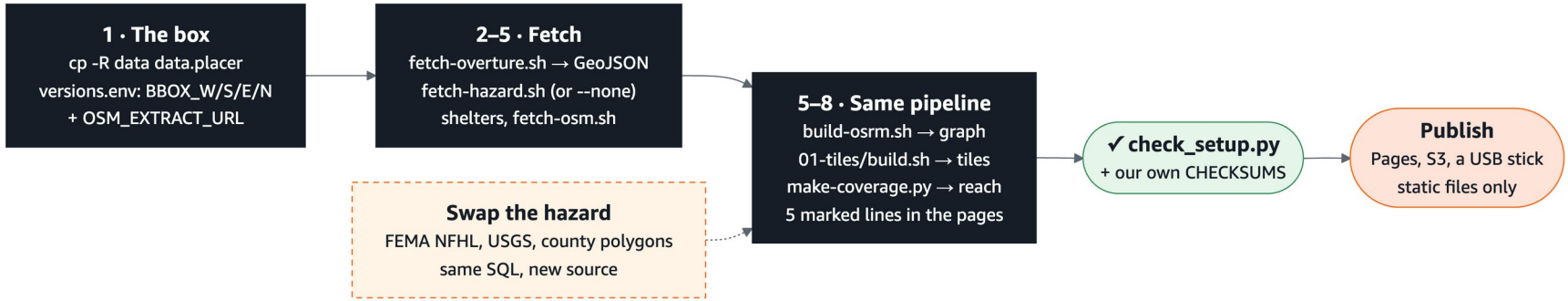
Adapt the pipeline to a community of our choice

one box → fetch → build → the same pages

versions.env | fetch-overture.sh | fetch-osm.sh | build-osrm.sh | build.sh | make-coverage.py

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

Phase 4 in one picture — one box, the same scripts, our own town



Phase 4 fetch-overture.sh → fetch-hazard.sh → shelters → fetch-osm.sh + build-osrm.sh → build.sh → make-coverage.py → 5 marked lines

Pinned DuckDB 2.0.0 | Planetiler 0.10.2 | pmtiles 1.31.2 | MapLibre GL JS 6.14 | OSRM v26.10.0

Reproducibility, then swap the community

If we can't rebuild it next year with the same answers, it's a demo, not a method.

`versions.env` pins every tool and image; `scripts/check-versions.sh` fails CI on drift

Dated extracts: Overture 2026-09-23.1, Geofabrik norcal 2026-10-22

`data/CHECKSUMS.sha256` verifies the 143 MB bundle we downloaded

One script per step — GeoJSON in, artifacts out:

fetch-overture.sh | fetch-hazard.sh | fetch-osm.sh

build-osrm.sh | 01-tiles/build.sh | make-coverage.py

PHASE 4 · Swap the community

1. `cp -R data data.placer` (keeps Placer), then in `versions.env` set `BBOX_W/S/E/N` and `OSM_EXTRACT_URL`

2. `scripts/fetch-overture.sh` (≈ 20 s), then `scripts/fetch-hazard.sh --none`

3. shelters ⟶ `fetch-osm.sh` + `build-osrm.sh` ⟶ `build.sh` ⟶ `make-coverage.py`

The OSM region download is the big one (100 MB – 1 GB); everything else runs in seconds.

Swap the hazard

FEMA flood zones instead of fire zones: polygons ⟶ ST_Intersects ⟶ tiles ⟶ match on a zone field. A data change, not a code change.

Going live (CAL FIRE perimeters feed) **raises the stakes on every caveat** we leave out: status, drive time, closures, capacity.

Laptop ⟶ planet: same shape, bigger inputs

Stage	Today (0.4 sq°)	At scale
Extract	DuckDB bbox pushdown, 24 s, six GeoJSON files (38 MB)	same SQL, a region at a time, run where the data lives
Tiles	Planetiler 4 s ⟶ 6.5 MB PMTiles	planet-scale builds are a documented Planetiler use case
Serve	npx serve / Caddy on :8080	PMTiles on object storage behind a CDN — still one file, no tile server
Routing	OSRM MLD graph 15 s, 99 MB	one graph per region; Valhalla / GraphHopper add isochrones and traffic
Freshness	pinned release 2026-09-23.1	Overture publishes monthly — bump the pin, rerun, diff the answers

In the wild

- OSM underlies OSRM — public server at router.project-osrm.org
- Planetiler builds OpenMapTiles-style and custom tile sets
- PMTiles is the Protomaps single-file format
- Overture releases monthly under the Linux Foundation
- MapLibre is the open fork lineage of Mapbox GL
- Real systems add live closures, shelter status, capacity and traffic. **We did not.**

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

WRAP-UP

Recap, licences, feedback

What we built → where it lives → how to ask for help

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

Recap — the pipeline we built



Open data ✦ open tools: no accounts, no licences to buy, no server to rent — all of it re-runs on a laptop.

Two questions, one extract: where are the safe places, and how do people reach them in time — a basemap and routes from the same data.

It answers them on a good day. It cannot tell anyone if today is a good day: shelter status, traffic, live closures and capacity are not in it.

Keep: the repo (MIT code, CC BY-SA 4.0 materials), the 143 MB data bundle (release data-v2), the public demo, and the habit: one box, one script per step.

Licences, attribution, links

What	Licence
Code	MIT
Materials (slides, docs)	CC BY-SA 4.0
OpenStreetMap	ODbL
Overture Maps	ODbL / CDLA-Permissive 2.0 (by theme)
CAL FIRE FHSZ, perimeters	CC-BY
ESA WorldCover	CC-BY 4.0
Noto Sans glyphs	SIL OFL 1.1

Repo github.com/vishal4c/routes-to-safety-workshop

Demo vishal4c.github.io/routes-to-safety-workshop/demo/

Data bundle .../releases/tag/data-v2 (143 MB; scripts/get-bundle.sh)

Feedback form [URL / QR to add]

Learn more

- duckdb.org/docs/extensions/spatial
- docs.overturemaps.org
- github.com/onthegomap/planetiler
- docs.protomaps.com/pmtiles
- maplibre.org
- project-osrm.org/docs

Attribution to keep on every map

- © OpenStreetMap contributors
- © Overture Maps Foundation
- Base map: © Esri, HERE, Garmin, USGS
- Fire Hazard Severity Zones: CAL FIRE FRAP (CC-BY)
- Land cover: ESA WorldCover (CC-BY)

Thank you.

Vishal Kumar and Shaishav Maisuria — questions: stay, or find us in the hallway.

Teaching example, not an official evacuation resource. Drive-time estimates assume no traffic and no closures.

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0